

Planora AI — Product Requirements Document

Further Development: Demo MVP -> Production-Grade Enterprise Platform

Founder: Abhijit · Prepared with Strategic AI Co-Founder

2026-07-05 · Version 1.0

Contents

1. Purpose & Scope	1
2. Product Vision & Objectives	2
3. Target Users & Personas	2
4. Guiding Principles & Non-Goals	3
5. Current Baseline (summary)	3
6. Release Plan Overview	3
7. Detailed Requirements by Epic	4
EPIC A — Multi-Tenancy & Data Isolation · R0 · P0	4
EPIC B — Authentication & Authorization · R0 · P0	4
EPIC C — Data Platform: Postgres, Migrations, Ingestion · R0–R1 · P0	4
EPIC D — Async Forecasting & Job Orchestration · R1 · P0	5
EPIC E — Primary Loop Productization · R1 · P0	5
EPIC F — Testing, CI/CD & Observability · R0–R2 · P0/P1	6
EPIC G — API Hardening & Security · R2 · P0/P1	6
EPIC H — AI Gateway & Agentic Planning (Differentiation) · R3 · P1	6
EPIC I — Expansion Modules · R4 · P1/P2	7
8. Non-Functional Requirements	7
9. Target Architecture	7
10. Data Model Requirements (Tenancy)	8
11. Success Metrics & Definition of Done	8
12. Milestones & Sequencing	8
13. Risks & Assumptions	9
14. Out of Scope (Frozen)	9

1. Purpose & Scope

This PRD defines the requirements to evolve Planora AI from its current state — a high-fidelity demo with one real forecasting engine on a single-tenant, unauthenticated prototype — into a production-grade, multi-tenant

enterprise SaaS platform.

It is the build-time companion to the **Planora AI Build Charter & Reality Matrix**. The Charter grades *what exists*; this PRD specifies *what to build, in what order, and how we know it's done*. Developers should treat the acceptance criteria in §7 as the contract for each piece of work.

In scope: platform foundation (tenancy, auth, data platform), productionizing the Primary planning loop, hardening (security, observability, testing), and the AI differentiation layer.

Out of scope for now (see §14): the [MOCK] demo-surface modules (BI query builder, simulated connectors, supplier portal, digital twin, pricing, retail, workforce). These are frozen until the foundation and Primary loop are production-ready.

2. Product Vision & Objectives

Vision: Planora AI is the planner-native decision-intelligence platform for the mid-market — delivering trusted demand forecasts and measurable working-capital impact in days, not the year-long deployments enterprise tools demand.

Objectives of this development cycle:

1. **Make it safe to sell.** Multi-tenant isolation and real server-side authentication so two customers can be onboarded without risk.
2. **Make one thing real end-to-end.** The Primary loop (Demand -> Diagnostics/FVA -> Inventory -> Consensus) runs on real customer data with zero mock values in its code path.
3. **Make it production-operable.** Tests, CI/CD, observability, async processing, and security hardening sufficient to pass an enterprise buyer's security review.
4. **Earn “AI-native.”** A server-side AI gateway and agentic planning loop that constitute a real, data-compounding moat — not a client-side API wrapper.

Success definition: a design partner uploads their own sales history and, within a day, receives real forecasts plus a quantified working-capital / stockout impact — on isolated, authenticated, tested infrastructure.

3. Target Users & Personas

Persona	Role	Primary goal	Key requirements this PRD serves
Demand Planner (Champion)	Owens forecasts day-to-day	Trustworthy forecasts, fast overrides	Primary loop, consensus workflow, FVA (EPIC E)
Supply/Inventory Planner	Manages stock & replenishment	Right stock, less trapped cash	Inventory optimization on real data (EPIC E)
VP Supply Chain / COO (Economic buyer)	Owens the P&L outcome	Service level + working capital	Exec analytics, ROI reporting (EPIC I)
IT / Security reviewer (Gatekeeper)	Approves the purchase	No security/compliance risk	Tenancy, auth, security posture (EPICS A, B, G)
Platform Admin (customer-side)	Manages users & settings	Control access & governance	RBAC, audit, admin console (EPIC B)

4. Guiding Principles & Non-Goals

Principles (enforced in code review):

- **Real-data-first.** No new feature ships with `hash()/random/mockData` in its runtime path. If it can't run on real tenant data, it isn't done.
- **Secure by default.** Every endpoint authenticated and tenant-scoped unless explicitly public.
- **Depth before breadth.** Harden the Primary loop before making any [MOCK] module real.
- **Tested math.** The forecasting/inventory engines — the crown jewels — are regression-locked before refactoring.
- **Honest docs.** No feature is described as “done”/“Strong” until it passes the §11 Definition of Done.

Non-goals for this cycle: new module breadth, mobile apps, marketplace, white-label, on-prem deployment, real-time streaming at scale.

5. Current Baseline (summary)

Per the Charter's verified findings: backend has **no authentication** (all 82 endpoints open; `passlib/jose` unused), **no multi-tenancy** (`orgId` hardcoded frontend-only), **no tests**, **no CI**, SQLite default with no migrations, Redis dependency unused, forecasting runs synchronously in-request. The forecasting/inventory/anomaly/clustering algorithms are genuinely real; most other module outputs are mocked. This PRD closes those gaps.

6. Release Plan Overview

Releases map 1:1 to Charter phases. Each release has an exit gate (§11); no release starts before the prior gate is met.

Release	Theme	Epics	Exit outcome
R0	Foundation	A, B, C(core), F(core)	Two isolated tenants, real logins, Postgres+migrations, CI green
R1	Real Primary Loop (Sellable MVP)	C(ingestion), D, E	Real forecasts + inventory impact on real data, async, mock-free
R2	Productionization	F(full), G	Passes security review; production load & monitoring
R3	Differentiation (AI moat)	H	Server-side AI gateway + agentic planning loop
R4	Expansion	I	S&OP/IBP + exec analytics real; land-and-expand ready

7. Detailed Requirements by Epic

Priority scale: **P0** = blocks release · **P1** = required for release quality · **P2** = desirable.

EPIC A — Multi-Tenancy & Data Isolation · R0 · P0

Goal: Every piece of data belongs to exactly one organization and is never visible to another.

Functional requirements:

- A-1. Introduce an `organizations` table and add `organization_id` (FK, indexed, non-null) to every business table (`demand_records`, `forecast_results`, `audit_logs`, `demand_sensing_signals`, `calendar_events`, `approval_requests`, `sku_master`, and all future tables).
- A-2. Enforce tenant scoping in a single shared data-access layer (tenant-scoped DB session or mandatory query filter), so no endpoint can accidentally read cross-tenant data.
- A-3. Tenant-scoped file/object storage for uploads (no shared filesystem paths).
- A-4. Tenant context derived from the authenticated principal (EPIC B), never from a client-supplied parameter.

Acceptance criteria:

- Given two orgs with data, when a user of Org A queries any endpoint, then only Org A data is ever returned — verified by an automated cross-tenant isolation test suite.
- Attempting to access another org's record by ID returns 404/403, never the record.
- A migration backfills existing data into a default org without loss.

EPIC B — Authentication & Authorization · R0 · P0

Goal: Real, server-enforced identity and permissions. Retire all mock auth.

Functional requirements:

- B-1. Server-side authentication: email/password with `passlib` bcrypt hashing (dependency already present); JWT access + refresh tokens issued and validated server-side.
- B-2. A `get_current_user` dependency applied to **every** non-public endpoint; unauthenticated requests are rejected.
- B-3. Server-enforced RBAC: move role->permission logic (`viewer/planner/manager/admin`) from the frontend into the API; frontend RBAC remains UX-only.
- B-4. Remove `MOCK_USERS` and hardcoded demo credentials from `AuthContext`; frontend authenticates against the real API.
- B-5. Password reset, session expiry/refresh, and account lockout on repeated failure.
- B-6. **Roadmap (P1, R2):** SSO/SAML and SCIM provisioning for enterprise.

Acceptance criteria:

- No endpoint returns data without a valid token (verified by an automated auth-coverage test asserting 401 on every route without credentials).
- A `viewer` cannot perform `edit:forecast`; enforcement is server-side and tested per role.
- No plaintext passwords exist anywhere in the codebase or database.

EPIC C — Data Platform: Postgres, Migrations, Ingestion · R0–R1 · P0

Goal: A real, migratable, validated data layer.

Functional requirements:

- C-1. PostgreSQL as the production database; SQLite permitted only for local dev.

- C-2. Alembic migrations for all schema changes from this point forward; no implicit `create_all` in production.
- C-3. Connection pooling and environment-based configuration.
- C-4. Re-evaluate JSON columns (`hierarchy_levels`, `exogenous_variables`) for query/index performance; normalize where they are filtered or aggregated.
- C-5. **Ingestion (R1)**: robust CSV/Excel upload mapping to the canonical schema with strict validation (types, required fields, date parsing, duplicate detection) and clear, row-level error reporting back to the user.
- C-6. **One real integration (R2)**: an SFTP/flat-file or mid-market API connector (e.g., NetSuite/QuickBooks tier) — explicitly **not** SAP.

Acceptance criteria:

- Schema changes are applied only via reviewed Alembic migrations; a fresh environment provisions from migrations alone.
- A malformed upload produces a precise, actionable error (which rows, which columns) and commits nothing.
- A valid upload of real customer data is queryable within the tenant with correct typing.

EPIC D — Async Forecasting & Job Orchestration · R1 · P0

Goal: Forecasting scales beyond a single SKU without HTTP timeouts.

Functional requirements:

- D-1. Job queue (Celery or RQ on the existing Redis dependency) for forecasting and other long-running compute.
- D-2. Batch forecasting across all SKUs in a dataset as an async job.
- D-3. Scheduled/recurring forecast runs.
- D-4. Job status (queued/running/succeeded/failed) surfaced in the UI with progress and error detail.
- D-5. Idempotency and retry semantics for failed jobs.

Acceptance criteria:

- A forecast run over a realistic multi-thousand-SKU dataset completes without request timeout and reports progress.
- A failed job surfaces a clear error and can be retried without duplicating results.

EPIC E — Primary Loop Productization · R1 · P0

Goal: Demand -> Diagnostics/FVA -> Inventory -> Consensus is real, end-to-end, mock-free — the sellable MVP.

Functional requirements:

- E-1. **Demand**: remove `mockData` from the demand module; forecasts run on real uploaded history; model selection/backtesting exposed with real MAE/RMSE/MAPE.
- E-2. **Forecast-vs-actual history**: persist forecasts and later actuals per tenant so accuracy and **FVA are computed from real data** (retire `mock_mape`).
- E-3. **Inventory**: safety stock, ROP/EOQ, ABC/XYZ, and working-capital outputs computed from the tenant's real demand + master data (not seeded).
- E-4. **Consensus**: multi-user consensus edits persisted per tenant with audit logging; override deltas tracked for FVA.
- E-5. **Onboarding**: a guided path to “time-to-first-forecast in under a day,” including sample-data option and validation feedback.
- E-6. **ROI output**: a clear, exportable summary of working capital freed / stockouts avoided / forecast-error reduction (the buyer-facing proof).

Acceptance criteria:

- A design partner uploads real history and receives real forecasts + inventory recommendations with no mock values in the code path (verified by code review + a “no-mock” lint check on the Primary-loop modules).
- FVA reflects real system-vs-human accuracy over time.
- Consensus edits by two users in the same org are correctly persisted, isolated, and audited.

EPIC F — Testing, CI/CD & Observability · R0–R2 · P0/P1

Goal: The platform is verifiable and operable.

Functional requirements:

- F-1. **(R0, P0)** pytest suite starting with forecasting/inventory math (regression-lock the crown jewels), plus the tenancy and auth isolation suites (EPICS A, B).
- F-2. **(R0, P0)** CI pipeline (e.g., GitHub Actions) blocking merges on failing tests/lint; coverage floor on Primary-loop paths.
- F-3. **(R1, P1)** Frontend tests (Vitest/Jest) + E2E (Playwright) for the Primary loop.
- F-4. **(R2, P1)** Observability: structured logging, error tracking (Sentry), request/latency metrics, health/readiness endpoints.
- F-5. **(R2, P1)** A “no-mock in production path” CI check that fails the build on `hash()/random/mockData` in graduated modules.

Acceptance criteria:

- CI is green and required for merge; the forecasting engine has regression tests that fail on numeric drift.
- Cross-tenant and unauthenticated-access tests exist and pass.
- Production errors are captured with context in Sentry.

EPIC G — API Hardening & Security · R2 · P0/P1

Goal: Survive an enterprise security review.

Functional requirements:

- G-1. API versioning (`/api/v1`), consistent pagination/filtering contract, Pydantic input validation on every endpoint.
- G-2. Rate limiting and abuse protection.
- G-3. Secrets management (env + vault; `.env.example` committed, no secrets in code); reviewed CORS policy.
- G-4. Tenant data encryption at rest and in transit; real, queryable audit trail.
- G-5. Security posture documentation and a **SOC 2-readiness** roadmap for buyer reviews.

Acceptance criteria:

- Every endpoint validates input and is rate-limited; no secret is present in the repository.
- A written security overview answers a standard enterprise security questionnaire.

EPIC H — AI Gateway & Agentic Planning (Differentiation) · R3 · P1

Goal: Make “AI-native” true and defensible.

Functional requirements:

- H-1. **Server-side AI gateway:** LLM keys live on the server; the client never holds a provider key. Retire browser-side BYO-key.
- H-2. **Data-compounding accuracy:** forecasting improves per tenant from accumulated history + planner overrides (FVA feedback loop).

- H-3. **Agentic planning:** an agent drafts the consensus forecast, flags exceptions, and proposes replenishment/PO actions for planner approval (decision automation, not chat).
- H-4. **Guardrails:** every AI-proposed action is auditable, explainable, and requires human approval by default.

Acceptance criteria:

- No LLM key is ever exposed to the browser.
- The agent produces an approvable draft plan with rationale that a planner accepts/edits; all actions are logged.

EPIC I — Expansion Modules · R4 · P1/P2

Goal: Turn [PARTIAL] modules real, in expansion-value order, only as paying customers demand them.

Functional requirements (high-level; each gets its own detailed PRD when scheduled):

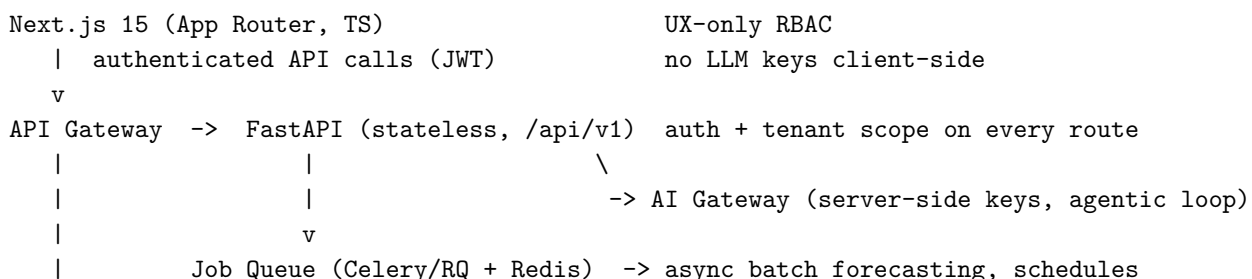
- I-1. **S&OP / IBP** as a real multi-user, stateful monthly-cycle workflow.
- I-2. **Executive Analytics / Control Tower** on real aggregated tenant data (retire `mockData`).
- I-3. Then, by demand: Pricing, Retail, Supplier Portal — each graduated from [PARTIAL]/[MOCK] to [REAL] via the §11 gate.

Acceptance criteria: each module passes the Definition of Done (§11) before being marketed as available.

8. Non-Functional Requirements

- **Security:** authenticated + tenant-scoped by default; encryption in transit and at rest; least-privilege RBAC; audit logging; SOC 2-readiness path.
- **Performance:** Primary-loop interactive responses < 2s p95; batch forecasting async with progress; no synchronous long compute in request threads.
- **Scalability:** stateless API workers horizontally scalable behind a load balancer; Redis caching for expensive reads; DB connection pooling.
- **Availability & reliability:** health/readiness probes; graceful failure and retries for jobs; documented backup/restore.
- **Data integrity:** validated ingestion; single source of truth for each metric (no contradictory numbers across modules).
- **Usability:** time-to-first-forecast < 1 day; clear error messaging; planner-native UX preserved.
- **Maintainability:** migrations for all schema change; tests required for merge; no mock in graduated code paths.

9. Target Architecture



v v
 PostgreSQL (Alembic migrations, org-scoped) Redis (cache + queue)
 Object storage (tenant-scoped uploads) Observability (Sentry, metrics, logs)

Principles: stateless workers, tenant isolation enforced centrally, long compute off the request path, AI inference server-owned.

10. Data Model Requirements (Tenancy)

- New **organizations** table (id, name, plan, created_at, settings).
- **users** table (id, organization_id FK, email unique-per-org, password_hash, role, status) — replacing frontend **MOCK_USERS**.
- **organization_id** FK (indexed, non-null) on every business table.
- All queries scoped by **organization_id** via the shared data layer.
- Alembic migration to introduce tenancy and backfill existing rows into a default organization.
- Reconsider JSON columns where filtered/aggregated; normalize hot paths.

11. Success Metrics & Definition of Done

Definition of Done (per feature/release):

- Runs on real tenant data with no mock/hash/random in the code path.
- Authenticated and tenant-isolated + RBAC enforced server-side.
- Covered by automated tests; CI green; core math regression-tested.
- Errors handled, logged, tracked; no silent failures.
- Long operations async with visible status.
- Input validated; bad data yields clear errors.
- Single source of truth for each metric.
- A new customer can be onboarded with no code changes.
- Documentation reflects reality (no aspirational claims).

Product success metrics to instrument now:

- Time-to-first-forecast (target < 1 day).
- Forecast-error reduction (MAPE lift) per tenant.
- Working capital freed / stockouts avoided per tenant (ROI proof).
- Pilot-to-paid conversion; logo count; early ARR.
- System reliability: uptime, job success rate, p95 latency.

12. Milestones & Sequencing

1. **R0 — Foundation:** EPIC A -> EPIC B -> EPIC C(core) -> EPIC F(core). *Gate:* two isolated tenants, real auth on every route, Postgres+migrations, CI green.
2. **R1 — Real Primary Loop:** EPIC C(ingestion) -> EPIC D -> EPIC E. *Gate:* real forecasts + inventory ROI on real data, async, mock-free; design partner onboardable in a day.
3. **R2 — Productionization:** EPIC F(full) -> EPIC G. *Gate:* passes enterprise security review; monitored production load.

4. **R3 — Differentiation:** EPIC H. *Gate:* server-side AI + approvable agentic plan.

5. **R4 — Expansion:** EPIC I. *Gate:* S&OP/IBP + exec analytics pass DoD.

Dependency rule: no epic starts before its prerequisites' gates are met. A, B, C precede everything.

13. Risks & Assumptions

Risk / Assumption	Impact	Mitigation
Open API exposed before auth ships	Critical	No external access until EPIC B gate met
Cross-tenant leak	Critical	EPIC A isolation tests are release-blocking
Forecasting refactor breaks the one real asset	High	Regression tests (F-1) before touching math
Breadth creep into [MOCK] modules early	High	\$14 freeze enforced in planning
Solo-founder bandwidth	High	Strict release gating; hire/co-founder for engineering
Beachhead assumption unvalidated	High	Confirm with design partners before over-building E-6

Assumptions: design partners will share real data; mid-market forecasting->inventory is the first pain worth paying for (validate early); Redis/Postgres/Celery are acceptable infra choices.

14. Out of Scope (Frozen)

Until R0 and R1 gates are met, do **not** invest engineering time in: BI query builder / semantic layer / data lineage, simulated ERP/WMS/TMS/procurement connectors, supplier portal, digital twin, pricing & promotion, retail & assortment, workforce planning, standalone financial planning. These remain available for demos but are not developed further this cycle. They re-enter scope, one at a time, via EPIC I only when a paying customer demands them.

Companion to the Planora AI Build Charter & Reality Matrix. Re-baseline this PRD at each release gate. Each EPIC I module receives its own detailed PRD when scheduled.